

```

p = &d3;                // d3 的类型是 Prot_Derv
p = &ddl;               // ddl 的类型是 Derived_from_Public
p = &dd2;               // dd2 的类型是 Derived_from_Private
p = &dd3;               // dd3 的类型是 Derived_from_Protected

```

练习 15.19: 假设 543 页和 544 页的每个类都有如下形式的成员函数:

```
void memfcn(Base &b) { b = *this; }
```

对于每个类, 分别判断上面的函数是否合法。

练习 15.20: 编写代码检验你对前面两题的回答是否正确。

练习 15.21: 从下面这些一般性抽象概念中任选一个 (或者选一个你自己的), 将其对应的一组类型组织成一个继承体系:

- (a) 图形文件格式 (如 gif、tiff、jpeg、bmp)
- (b) 图形基元 (如方格、圆、球、圆锥)
- (c) C++语言中的类型 (如类、函数、成员函数)

练习 15.22: 对于你在上一题中选择的类, 为其添加合适的虚函数及公有成员和受保护的成员。

## 15.6 继承中的类作用域



每个类定义自己的作用域 (参见 7.4 节, 第 253 页), 在这个作用域内我们定义类的成员。当存在继承关系时, 派生类的作用域嵌套 (参见 2.2.4 节, 第 43 页) 在其基类的作用域之内。如果一个名字在派生类的作用域内无法正确解析, 则编译器将继续在外层的基类作用域中寻找该名字的定义。

派生类的作用域位于基类作用域之内这一事实可能有点儿出人意料, 毕竟在我们的程序文本中派生类和基类的定义是相互分离开来的。不过也恰恰因为类作用域有这种继承嵌套的关系, 所以派生类才能像使用自己的成员一样使用基类的成员。例如, 当我们编写下面的代码时:

```

Bulk_quote bulk;
cout << bulk.isbn();

```

名字 isbn 的解析将按照下述过程所示:

- 因为我们是通过 Bulk\_quote 的对象调用 isbn 的, 所以首先在 Bulk\_quote 中查找, 这一步没有找到名字 isbn。
- 因为 Bulk\_quote 是 Disc\_quote 的派生类, 所以接下来在 Disc\_quote 中查找, 仍然找不到。
- 因为 Disc\_quote 是 Quote 的派生类, 所以接着查找 Quote; 此时找到了名字 isbn, 所以我们使用的 isbn 最终被解析为 Quote 中的 isbn。

### 在编译时进行名字查找

一个对象、引用或指针的静态类型 (参见 15.2.3 节, 第 532 页) 决定了该对象的哪些成员是可见的。即使静态类型与动态类型可能不一致 (当使用基类的引用或指针时会发生

这种情况), 但是我们能使用哪些成员仍然是由静态类型决定的。举个例子, 我们可以给 `Disc_quote` 添加一个新成员, 该成员返回一个存有最小 (或最大) 数量及折扣价格的 `pair` (参见 11.2.3 节, 第 379 页):

```
class Disc_quote : public Quote {
public:
    std::pair<size_t, double> discount_policy() const
    { return {quantity, discount}; }
    // 其他成员与之前的版本一致
};
```

我们只能通过 `Disc_quote` 及其派生类的对象、引用或指针使用 `discount_policy`:

```
Bulk_quote bulk;
Bulk_quote *bulkP = &bulk;           // 静态类型与动态类型一致
Quote *itemP = &bulk;                 // 静态类型与动态类型不一致
bulkP->discount_policy();               // 正确: bulkP 的类型是 Bulk_quote*
itemP->discount_policy();               // 错误: itemP 的类型是 Quote*
```

**618** 尽管在 `bulk` 中确实含有一个名为 `discount_policy` 的成员, 但是该成员对于 `itemP` 却是不可见的。`itemP` 的类型是 `Quote` 的指针, 意味着对 `discount_policy` 的搜索将从 `Quote` 开始。显然 `Quote` 不包含名为 `discount_policy` 的成员, 所以我们无法通过 `Quote` 的对象、引用或指针调用 `discount_policy`。

## 名字冲突与继承

和其他作用域一样, 派生类也能重用定义在其直接基类或间接基类中的名字, 此时定义在内层作用域 (即派生类) 的名字将隐藏定义在外层作用域 (即基类) 的名字 (参见 2.2.4 节, 第 43 页):

```
struct Base {
    Base(): mem(0) { }
protected:
    int mem;
};
struct Derived : Base {
    Derived(int i): mem(i) { }           // 用 i 初始化 Derived::mem
                                         // Base::mem 进行默认初始化
    int get_mem() { return mem; }       // 返回 Derived::mem
protected:
    int mem;                             // 隐藏基类中的 mem
};
```

`get_mem` 中 `mem` 引用的解析结果是定义在 `Derived` 中的名字, 下面的代码

```
Derived d(42);
cout << d.get_mem() << endl;           // 打印 42
```

的输出结果将是 42。

*Note*

派生类的成员将隐藏同名的基类成员。

## 通过作用域运算符来使用隐藏的成员

我们可以通过作用域运算符来使用一个被隐藏的基类成员:

```
struct Derived : Base {
    int get_base_mem() { return Base::mem; }
    // ...
};
```

作用域运算符将覆盖掉原有的查找规则,并指示编译器从 Base 类的作用域开始查找 mem。如果使用最新的 Derived 版本运行上面的代码,则 d.get\_mem() 的输出结果将是 0。

**Best Practices**

除了覆盖继承而来的虚函数之外,派生类最好不要重用其他定义在基类中的名字。

### 关键概念: 名字查找与继承

619

理解函数调用的解析过程对于理解 C++ 的继承至关重要,假定我们调用 p->mem() (或者 obj.mem()),则依次执行以下 4 个步骤:

- 首先确定 p (或 obj) 的静态类型。因为我们调用的是一个成员,所以该类型必然是类类型。
- 在 p (或 obj) 的静态类型对应的类中查找 mem。如果找不到,则依次在直接基类中不断查找直至到达继承链的顶端。如果找遍了该类及其基类仍然找不到,则编译器将报错。
- 一旦找到了 mem,就进行常规的类型检查(参见 6.1 节,第 183 页)以确认对于当前找到的 mem,本次调用是否合法。
- 假设调用合法,则编译器将根据调用的是否是虚函数而产生不同的代码:
  - 如果 mem 是虚函数且我们是通过引用或指针进行的调用,则编译器产生的代码将在运行时确定到底运行该虚函数的哪个版本,依据是对象的动态类型。
  - 反之,如果 mem 不是虚函数或者我们是通过对象(而非引用或指针)进行的调用,则编译器将产生一个常规函数调用。

### 一如往常,名字查找先于类型检查

如前所述,声明在内层作用域的函数并不会重载声明在外层作用域的函数(参见 6.4.1 节,第 210 页)。因此,定义派生类中的函数也不会重载其基类中的成员。和其他作用域一样,如果派生类(即内层作用域)的成员与基类(即外层作用域)的某个成员同名,则派生类将在其作用域内隐藏该基类成员。即使派生类成员和基类成员的形参列表不一致,基类成员也仍然会被隐藏掉:

```
struct Base {
    int memfcn();
};
struct Derived : Base {
    int memfcn(int);           // 隐藏基类的 memfcn
};
Derived d; Base b;
b.memfcn();                   // 调用 Base::memfcn
d.memfcn(10);                 // 调用 Derived::memfcn
d.memfcn();                   // 错误: 参数列表为空的 memfcn 被隐藏了
d.Base::memfcn();             // 正确: 调用 Base::memfcn
```

Derived 中的 memfcn 声明隐藏了 Base 中的 memfcn 声明。在上面的代码中前两条调用语句容易理解，第一个通过 Base 对象 b 进行的调用执行基类的版本；类似的，第二个通过 d 进行的调用执行 Derived 的版本；第三条调用语句有点特殊，d.memfcn() 是非法的。

620

为了解析这条调用语句，编译器首先在 Derived 中查找名字 memfcn；因为 Derived 确实定义了一个名为 memfcn 的成员，所以查找过程终止。一旦名字找到，编译器就不再继续查找了。Derived 中的 memfcn 版本需要一个 int 实参，而当前的调用语句无法提供任何实参，所以该调用语句是错误的。



## 虚函数与作用域

我们现在可以理解为什么基类与派生类中的虚函数必须有相同的形参列表了（参见 15.3 节，第 537 页）。假如基类与派生类的虚函数接受的实参不同，则我们就无法通过基类的引用或指针调用派生类的虚函数了。例如：

```
class Base {
public:
    virtual int fcn();
};
class D1 : public Base {
public:
    // 隐藏基类的 fcn，这个 fcn 不是虚函数
    // D1 继承了 Base::fcn() 的定义
    int fcn(int);                // 形参列表与 Base 中的 fcn 不一致
    virtual void f2();           // 是一个新的虚函数，在 Base 中不存在
};
class D2 : public D1 {
public:
    int fcn(int);                // 是一个非虚函数，隐藏了 D1::fcn(int)
    int fcn();                   // 覆盖了 Base 的虚函数 fcn
    void f2();                   // 覆盖了 D1 的虚函数 f2
};
```

D1 的 fcn 函数并没有覆盖 Base 的虚函数 fcn，原因是它们的形参列表不同。实际上，D1 的 fcn 将隐藏 Base 的 fcn。此时拥有了两个名为 fcn 的函数：一个是 D1 从 Base 继承而来的虚函数 fcn；另一个是 D1 自己定义的接受一个 int 参数的非虚函数 fcn。

## 通过基类调用隐藏的虚函数

给定上面定义的这些类后，我们来看几种使用其函数的方法：

```
Base bobj; D1 d1obj; D2 d2obj;

Base *bp1 = &bobj, *bp2 = &d1obj, *bp3 = &d2obj;
bp1->fcn();                // 虚调用，将在运行时调用 Base::fcn
bp2->fcn();                // 虚调用，将在运行时调用 Base::fcn
bp3->fcn();                // 虚调用，将在运行时调用 D2::fcn

D1 *d1p = &d1obj; D2 *d2p = &d2obj;
bp2->f2();                  // 错误：Base 没有名为 f2 的成员
d1p->f2();                  // 虚调用，将在运行时调用 D1::f2()
d2p->f2();                  // 虚调用，将在运行时调用 D2::f2()
```

前三条调用语句是通过基类的指针进行的，因为 `fcn` 是虚函数，所以编译器产生的代码将在运行时确定使用虚函数的哪个版本。判断的依据是该指针所绑定对象的真实类型。在 `bp2` 的例子中，实际绑定的对象是 `D1` 类型，而 `D1` 并没有覆盖那个不接受实参的 `fcn`，所以通过 `bp2` 进行的调用将在运行时解析为 `Base` 定义的版本。

621

接下来的三条调用语句是通过不同类型的指针进行的，每个指针分别指向继承体系中的一个类型。因为 `Base` 类中没有 `f2()`，所以第一条语句是非法的，即使当前的指针碰巧指向了一个派生类对象也无济于事。

为了完整地阐明上述问题，我们不妨再观察一些对于非虚函数 `fcn(int)` 的调用语句：

```
Base *p1 = &d2obj; D1 *p2 = &d2obj; D2 *p3 = &d2obj;
p1->fcn(42);           // 错误：Base 中没有接受一个 int 的 fcn
p2->fcn(42);           // 静态绑定，调用 D1::fcn(int)
p3->fcn(42);           // 静态绑定，调用 D2::fcn(int)
```

在上面的每条调用语句中，指针都指向了 `D2` 类型的对象，但是由于我们调用的是非虚函数，所以不会发生动态绑定。实际调用的函数版本由指针的静态类型决定。

## 覆盖重载的函数

和其他函数一样，成员函数无论是否是虚函数都能被重载。派生类可以覆盖重载函数的 0 个或多个实例。如果派生类希望所有的重载版本对于它来说都是可见的，那么它就需要覆盖所有的版本，或者一个也不覆盖。

有时一个类仅需覆盖重载集合中的一些而非全部函数，此时，如果我们不得不覆盖基类中的每一个版本的话，显然操作将极其烦琐。

一种好的解决方案是为重载的成员提供一条 `using` 声明语句（参见 15.5 节，第 546 页），这样我们就无须覆盖基类中的每一个重载版本了。`using` 声明语句指定一个名字而不指定形参列表，所以一条基类成员函数的 `using` 声明语句就可以把该函数的所有重载实例添加到派生类作用域中。此时，派生类只需要定义其特有的函数就可以了，而无须为继承而来的其他函数重新定义。

类内 `using` 声明的一般规则同样适用于重载函数的名字（参见 15.5 节，第 546 页）；基类函数的每个实例在派生类中都必须是可访问的。对派生类没有重新定义的重载版本的访问实际上是对 `using` 声明点的访问。

## 15.6 节练习

**练习 15.23：**假设第 550 页的 `D1` 类需要覆盖它继承而来的 `fcn` 函数，你应该如何对其进行修改？如果你修改之后 `fcn` 匹配了 `Base` 中的定义，则该节的那些调用语句将如何解析？

## 15.7 构造函数与拷贝控制

622

和其他类一样，位于继承体系中的类也需要控制当其对象执行一系列操作时发生什么样的行为，这些操作包括创建、拷贝、移动、赋值和销毁。如果一个类（基类或派生类）没有定义拷贝控制操作，则编译器将为它合成一个版本。当然，这个合成的版本也可以定义成被删除的函数。